

day 24 | 251024 F

herein

We have worked on Euler's method and have a decent way to do that. But there are other methods that work even better and don't take that much more calculation to do. These methods all go by the name Runge-Kutta. Newton's method is actually 1st order Runge-Kutta. 2nd order Runge-Kutta and 4th order Runge-Kutta are what we are going to talk about today.

hence

I need a better explanation of Runge-Kutta to show the students. I need to feel intuitive and also easy to understand the extensions to it. But that is a thing for next time. Also I need to start off with a better function with a known answer so that we can compare how much better the Runge-Kutta is to the Euler's method.

```
In [41]: %matplotlib ipynb
import numpy as np
import matplotlib.pyplot as plt
```

```
In [52]: # Euler's Method

# define the differential equation
def f(x_, t_): # dx/dt
    # return(-x_**3+np.sin(t_))
    return(t_**2+t_-1)

# define the boundary condition
a = 0.0
b = 10.0
N = 50
dt = (b-a)/N

x = 1.0 # initial condition

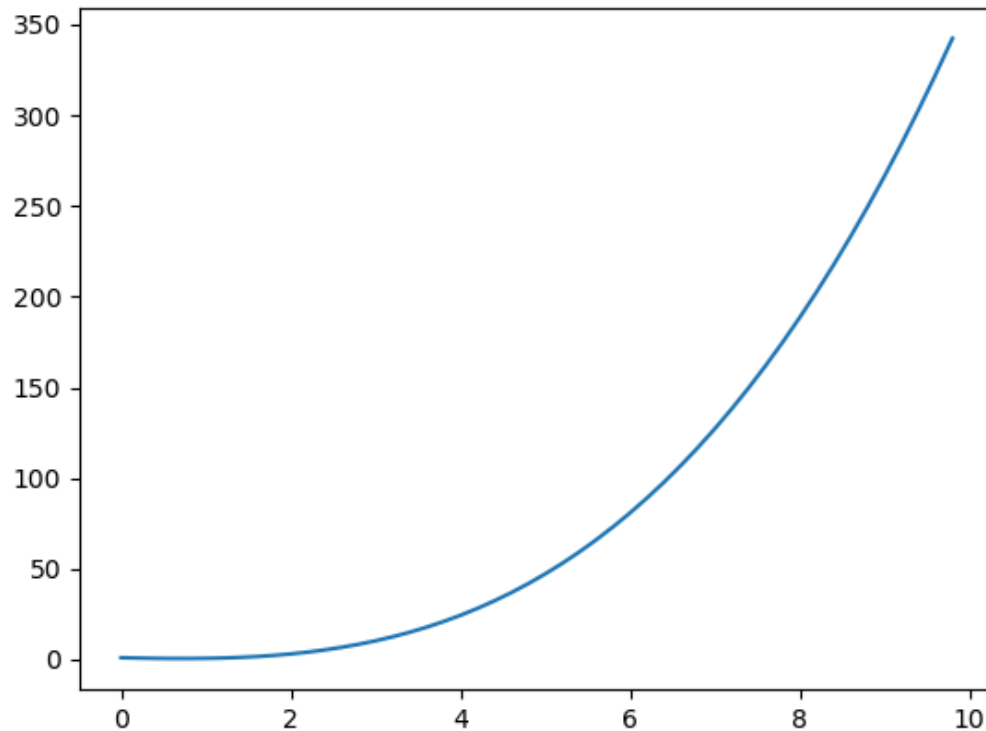
tpoints = np.arange(a, b, dt)
xpointsE = []

for i in tpoints:
    xpointsE.append(x)
    x = x + f(x, i) * dt

fig0, ax0 = plt.subplots()
ax0.plot(tpoints, xpointsE)
```

Out[52]: [<matplotlib.lines.Line2D at 0x7d1c29ec7ec0>]

Figure



```
In [53]: # Runge-Kutta 2nd order

x = 1.0 # initial condition

tpoints = np.arange(a, b, dt)
xpointsRK2 = []

for i in tpoints:
    xpointsRK2.append(x)
    k1 = dt*f(x, i)
    k2 = dt*f(x+1/2*k1, i+1/2*dt)
    x = x + k2
```

```
In [54]: # Runge-Kutta 4th order

x = 1.0 # initial condition

tpoints = np.arange(a, b, dt)
xpointsRK4 = []

for i in tpoints:
    xpointsRK4.append(x)
    k1 = dt*f(x, i)
    k2 = dt*f(x+1/2*k1, i+1/2*dt)
    k3 = dt*f(x+1/2*k2, i+1/2*dt)
```

```

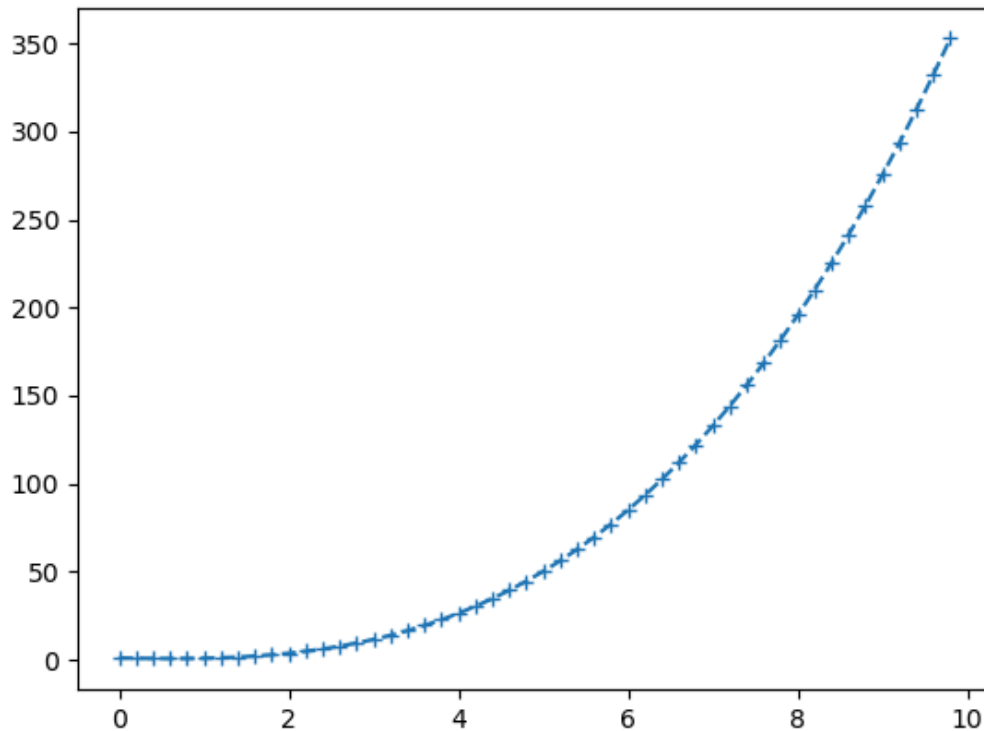
k4 = dt*f(x+k3, i+dt)
x = x + (k1+2*k2+2*k3+k4)/6

fig2, ax2 = plt.subplots()
ax2.plot(tpoints, xpointsRK4, '+--')

```

Out[54]: [<matplotlib.lines.Line2D at 0x7d1c29cbaf30>]

Figure



At this points I let the students work on plotting these all on the same chart. They needed to change the names of the outputs and then get them to work and plot them all on the same chart. Below is a fragmented example of how they should do this, which I gave them after some time working on it in groups.

```

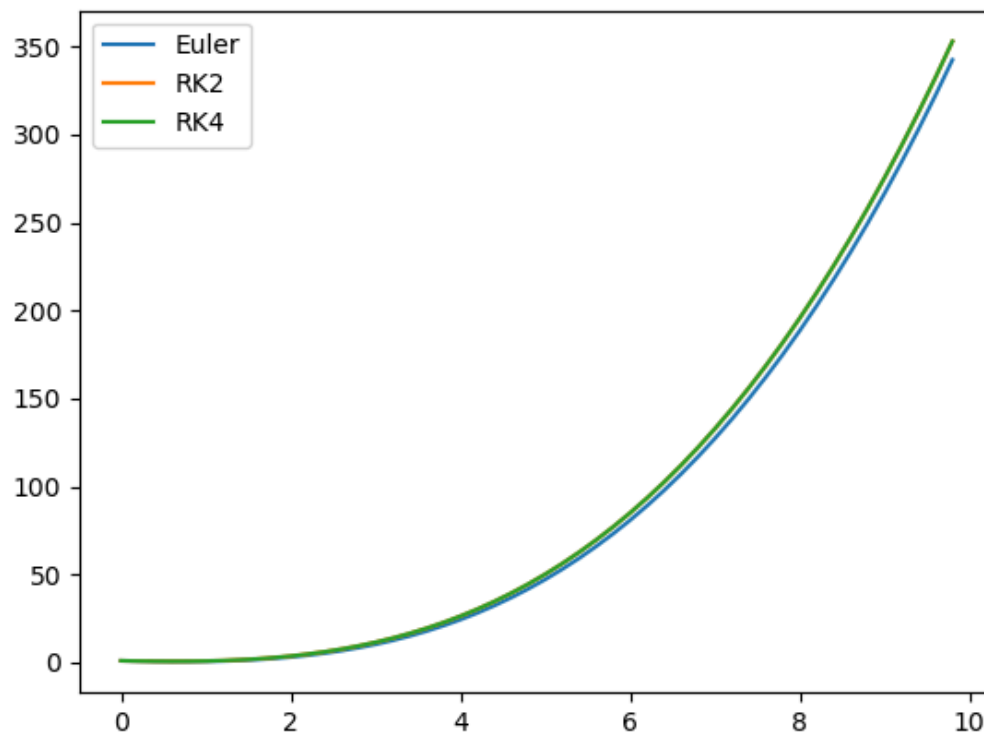
In [55]: fig, ax = plt.subplots()
ax.plot(tpoints, xpointsE, label='Euler')
ax.plot(tpoints, xpointsRK2, label='RK2')
ax.plot(tpoints, xpointsRK4, label='RK4')

ax.legend()

```

Out[55]: <matplotlib.legend.Legend at 0x7d1c29d37c20>

Figure



In []:

In [77]: `from math import *`

```
In [92]: # define the differential equation
def f(x_, t_): # dx/dt
    # return(-x_**3+np.sin(t_))
    return(x_+t_)

# define the boundary condition
a = 0.0
b = 10.0
N = 20
dt = (b-a)/N

x=0
x0 = 1

# Euler's Method

x = x0 # initial condition

tpoints = np.arange(a, b, dt)
xpointsE = []

for i in tpoints:
```

```

    xpointsE.append(x)
    x = x + f(x, i) * dt

# Runge-Kutta 2nd order

x = x0 # initial condition

tpoints = np.arange(a, b, dt)
xpointsRK2 = []

for i in tpoints:
    xpointsRK2.append(x)
    k1 = dt*f(x, i)
    k2 = dt*f(x+1/2*k1, i+1/2*dt)
    x = x + k2

# Runge-Kutta 4th order

x = x0 # initial condition

tpoints = np.arange(a, b, dt)
xpointsRK4 = []

for i in tpoints:
    xpointsRK4.append(x)
    k1 = dt*f(x, i)
    k2 = dt*f(x+1/2*k1, i+1/2*dt)
    k3 = dt*f(x+1/2*k2, i+1/2*dt)
    k4 = dt*f(x+k3, i+dt)
    x = x + (k1+2*k2+2*k3+k4)/6

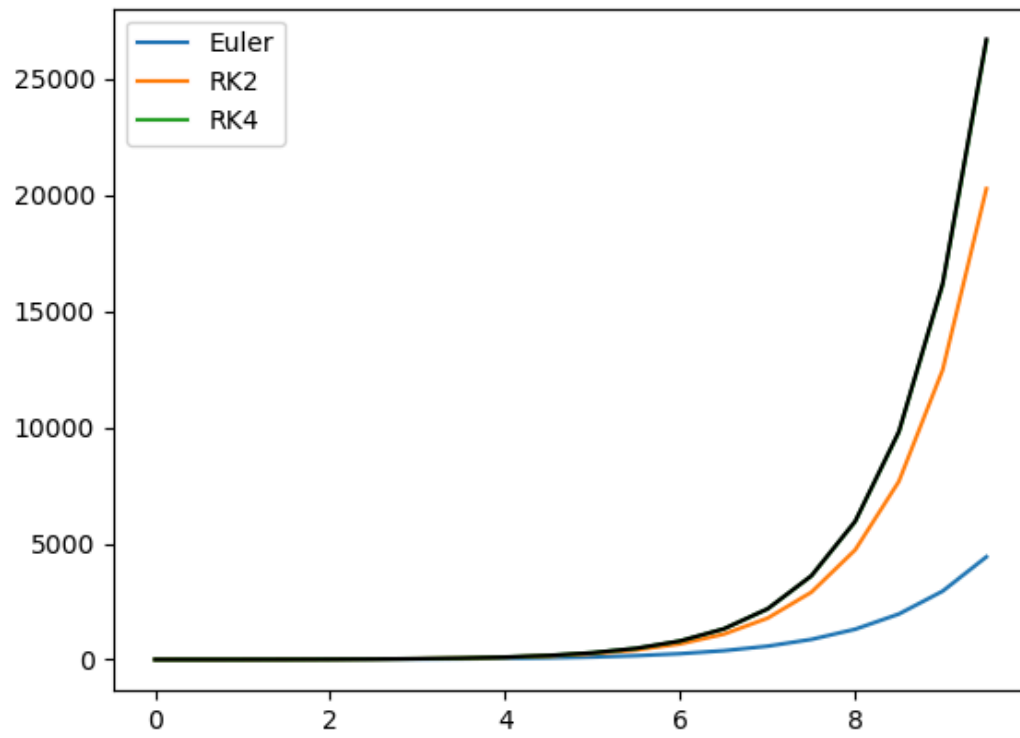
fig, ax = plt.subplots()
ax.plot(tpoints, xpointsE, label='Euler')
ax.plot(tpoints, xpointsRK2, label='RK2')
ax.plot(tpoints, xpointsRK4, label='RK4')
ax.plot(tpoints, 2*np.exp(tpoints)-tpoints-1, 'k')

ax.legend()

```

Out[92]: <matplotlib.legend.Legend at 0x7d1c236d7dd0>

Figure



In []:

In []: